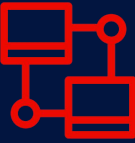


Text and Image Generation with Generative AI

CSIT205 Generative AI – 2025



UNIVERSITY
OF WOLLONGONG
AUSTRALIA



Wollongong
Dr. Rory Sie
(rorys@uow.edu.au)

*SCHOOL OF COMPUTING AND INFORMATION
TECHNOLOGY*

*FACULTY OF ENGINEERING AND INFORMATION
SCIENCES*



UNIVERSITY
OF WOLLONGONG
AUSTRALIA



Acknowledgement of Country

We acknowledge that Country for Aboriginal peoples is an interconnected set of ancient and sophisticated relationships.

The University of Wollongong (UOW) spreads across many interrelated Aboriginal Countries that are bound by this sacred landscape, and intimate relationship with that landscape since creation. From Sydney to the Southern Highlands, to the South Coast. From fresh water to bitter water to salt. From city to urban to rural.

The University of Wollongong acknowledges the custodianship of the Aboriginal peoples of this place and space that has kept alive the relationships between all living things. The University acknowledges the devastating impact of colonisation on our campuses' footprint and commit ourselves to truth-telling, healing and education.

Journey Artwork by Wodi Wodi
Artist Lauren Henry.

The background is a vibrant rainbow with horizontal bands of red, orange, yellow, green, cyan, blue, and purple. Large, dark blue letters 'U', 'O', and 'W' are positioned on the right side of the image, with the 'U' at the top, 'O' in the middle, and 'W' at the bottom.

Everyone is welcome here.

UOW takes pride in
the diversity of our
community.

Find out more, seek support or join
the UOW Ally Network
uow.info/ally

Subject introduction



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Description

In this subject, students delve into the world of Generative AI. They explore the foundational principles and practical applications of this cutting-edge field. Through a blend of theory and hands-on experience, students gain insights into how Generative AI empowers artificial intelligence to create original content – ranging from text and images to video and code – based on user prompts. The curriculum covers essential topics, including AI's role in code generation, effective prompt engineering techniques, and the development of advanced generative models like GANs and Transformers. Moreover, students critically examine the ethical implications of Generative AI and its impact on businesses strategy. By the end of the subject, students will have a comprehensive understanding of this promising field and the ability to responsibly apply Generative AI in real-world scenarios.

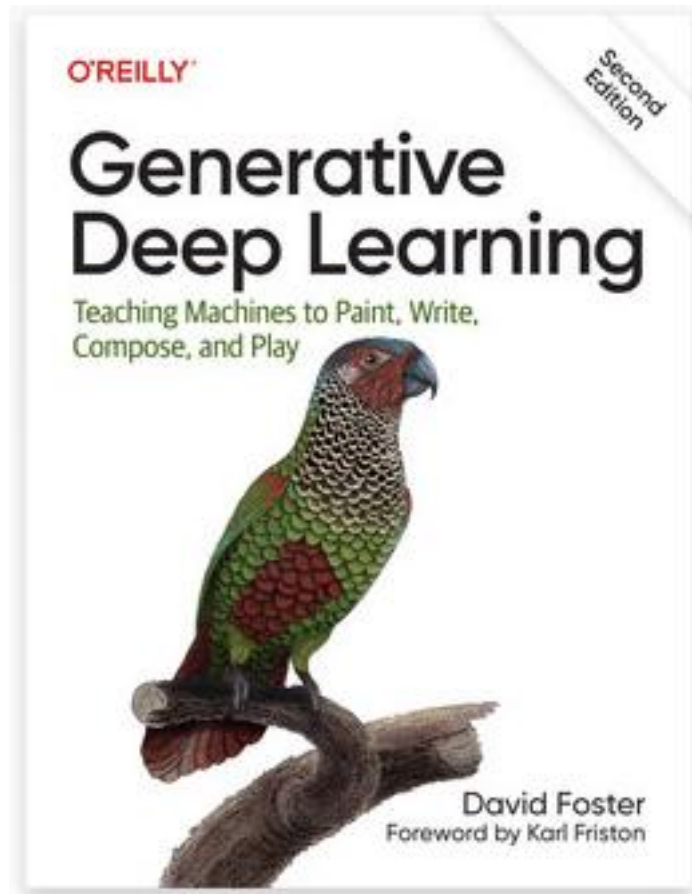
Subject Learning Outcomes (SLOs)

On successful completion of this subject, students will be able to:

1. Demonstrate an understanding of the foundations of Generative AI and its practical applications in text generation, image generation and assisted coding.
2. Apply effective prompt engineering for various tasks.
3. Demonstrate an understanding of techniques to enhance model performance, such as fine-tuning, Retrieval Augmented Generation and Reinforcement Learning using Human Feedback.
4. Identify the impact of Generative AI for industry and employ effective strategies to incorporate Generative AI.
5. Demonstrate ethical and responsible use of Generative AI.

Don't forget 

This is the book we're using



<https://www.oreilly.com/library/view/generative-deep-learning/978109813>

Assessment 1

- Deliverables (For both reports, write your full name and UOW ID at the top of first page)
- File name must be in the form of: TXX_NAME_UOWID.txt for prompt-answer pairs where XX is your tutorial group, NAME is your full name (without space or underscore) and UOWID is your 7-digit UOW ID number (not SIM student ID number).

For example, T02_BobChew_8080426.txt

Assessment 1

- Deliverables (For both reports, write your full name and UOW ID at the top of first page)
- File name must be in the form of: TXX_NAME_UOWID.doc for report where XX is your tutorial group, NAME is your full name (without space or underscore) and UOWID is your 7-digit UOW ID number (not SIM student ID number).
For example, T02_BobChew_8080426.doc

Assessment 2

Assessment 2 – Generative AI business case

- (Day 1 – Form a group of 5 or 6 students and select a responsible student as a group leader. The group leader of a tutorial class is to inform the lecturer of the tutorial class, group members UOW id and names before end of 8th day.)
- (All team members must be present unless there is a valid reason during the interview with CTO during lab 3 and presentation during lab 4.)

Assessment 2

Assessment 2 – Generative AI business case

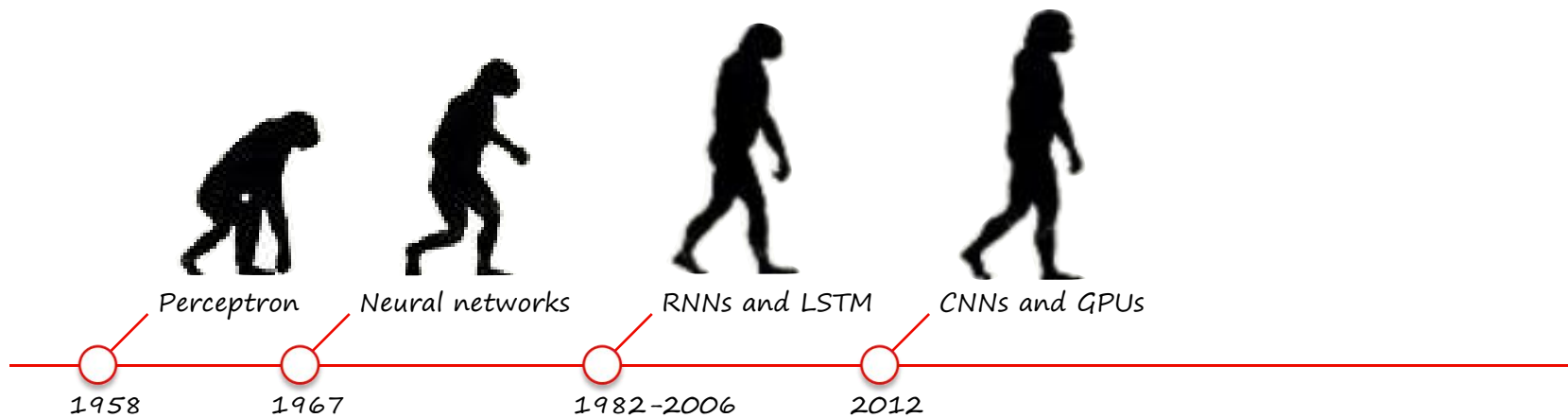
- File name must be in the form of: TXX_NAME_UOWID.doc where XX is your tutorial group, NAME is the group leader full name (without space or underscore) and UOWID is your 7-digit UOW ID number (not SIM student ID number).
For example, TO2_BobChew_8080426.doc
- (For the report and presentation slides, write the full name of each group member and their UOW ID at the top of first page)

What did we discuss last week?



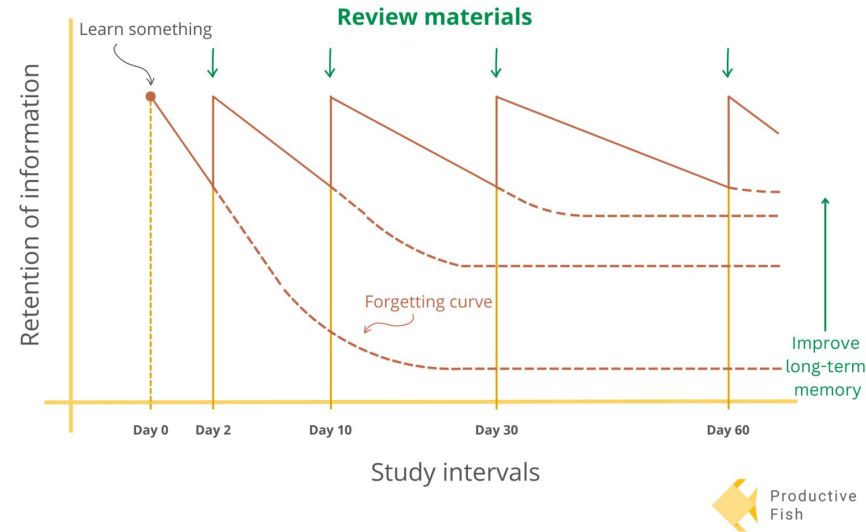
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Overview lecture 1



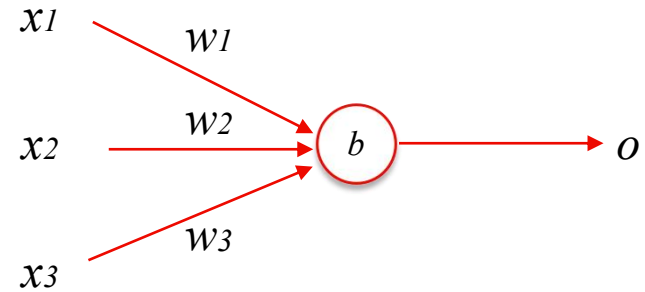
Why are we repeating things in the lecture and labs?

- Why? Your brain is like a muscle 🦵 it grows (read: learns) at rest
 - This is why cramming does not work
- How? Learn - space - repeat - space - repeat
 - Vary in space size
 - Longer spaces generally work better; 1 hour is enough
 - Make sure to do something completely different during the 'spaces'



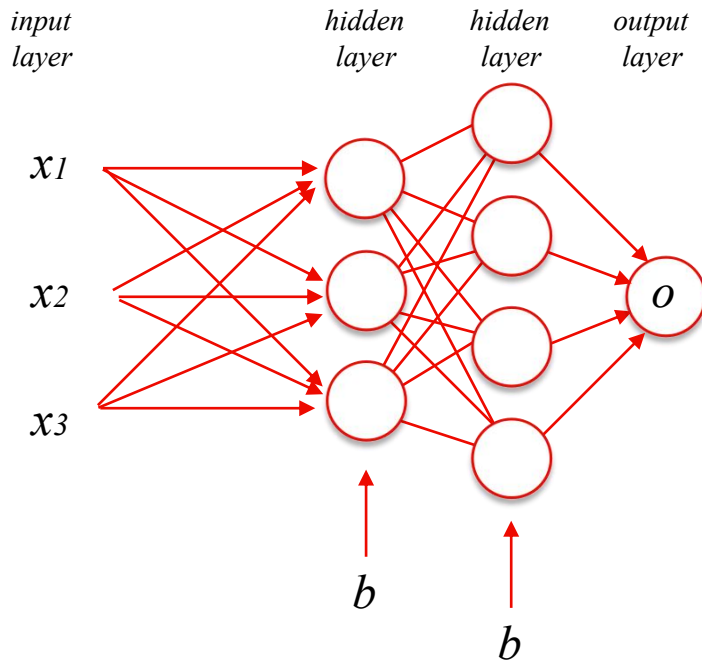
Perceptron - how it works

- Linear function
 - Several inputs x
 - Weights w
 - Bias b
 - One output o
- Learns by updating its weights w and bias b
- Supervised learning



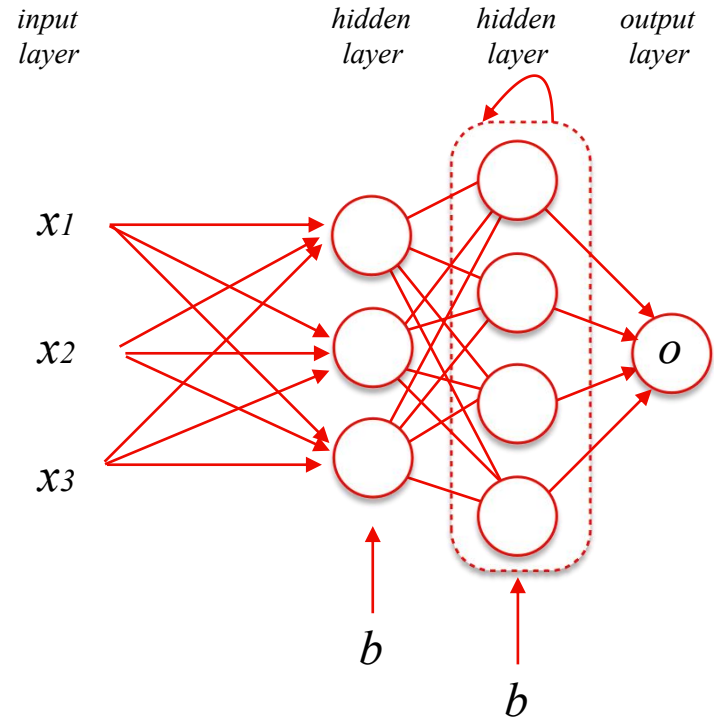
Neural networks

- Basically a perceptron with more hidden layers:
 - One input layer
 - One output layer
 - Multiple hidden layers
- Weights (edges) and biases between layers are updated while learning
- Calculations go from left to right ('feed-forward neural network')



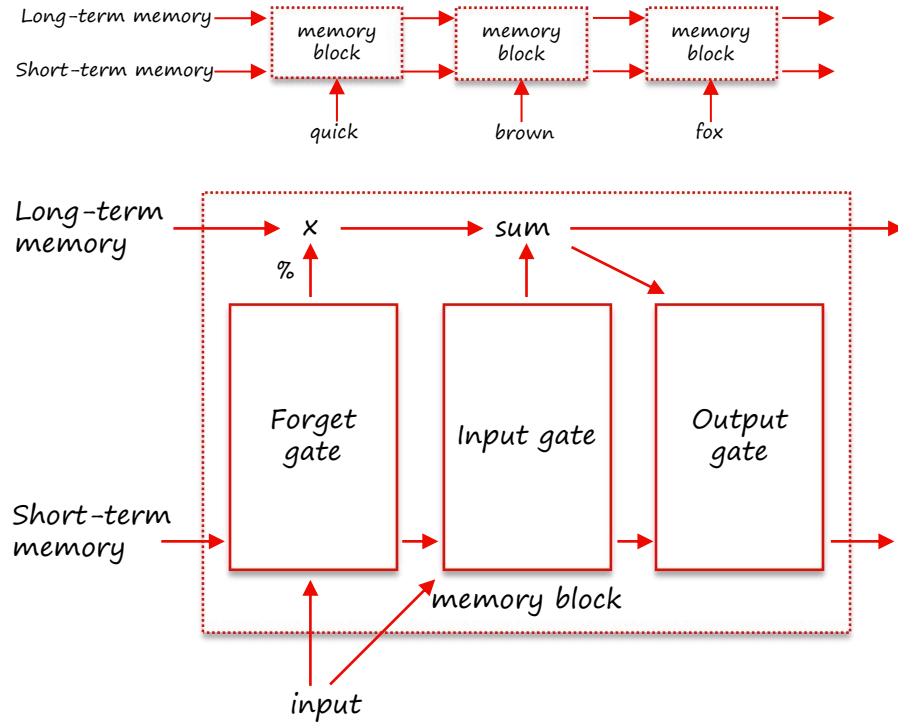
Recurrent Neural networks

- Basically NNs that can have loops to / within their own layer
- So they can model relationships forward and backward in time (e.g. sequences): “The quick brown fox jumps over the lazy dog”



Long Short-Term Memory (LSTM)

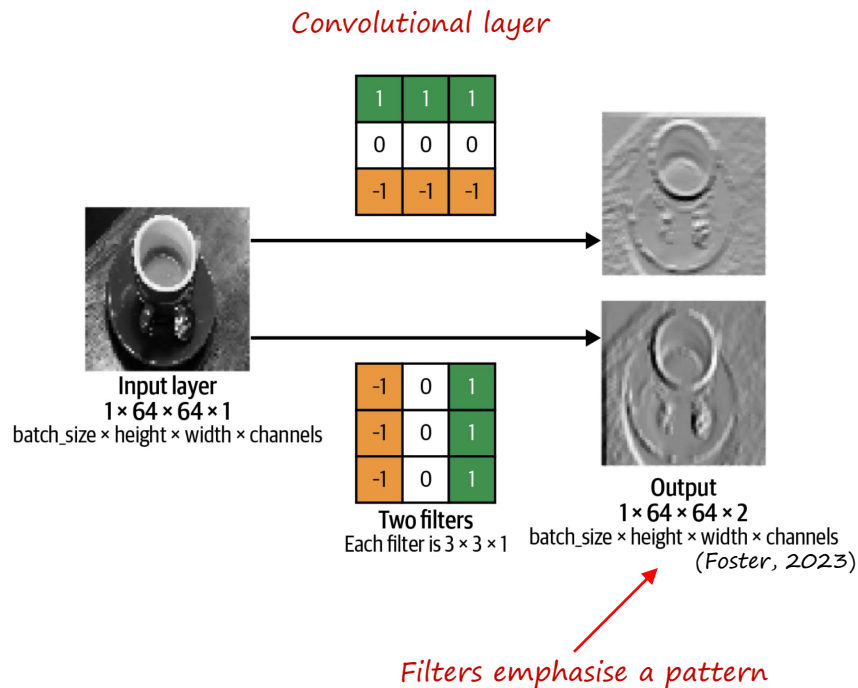
- Type of RNN
- Memory blocks instead of neurons
- Employs a forget gate that transforms input between 0 and 1, which means it can retain important information
- → addresses the vanishing gradient and long term dependency problems



More info: <https://youtu.be/YCzL96nL7j>

Convolutional Neural Networks (CNNs)

- 2012: AlexNet (Krizhevsky et al., 2012) wins ImageNet competition.
- Analyses grid-like topologies, like images.
- Architecture:
 - Convolutional layers that apply kernels / filters to detect features
 - Pooling layers (not shown) to reduce dimensionality

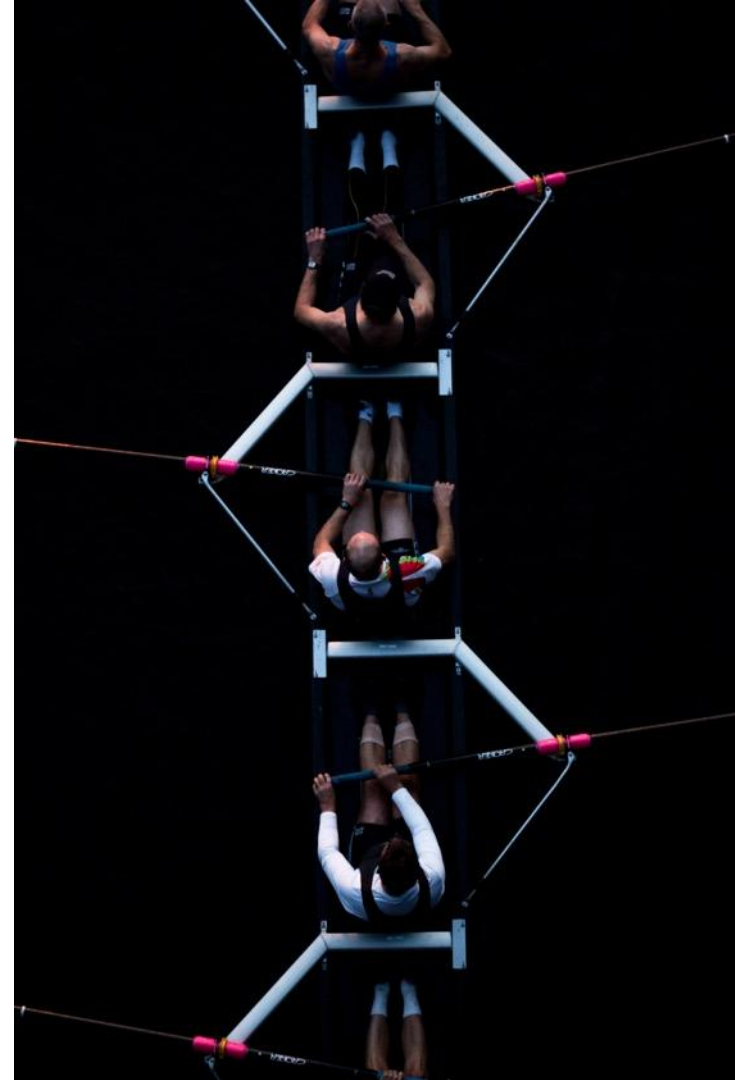


Why are we here?

CSIT205 Generative AI - Module 1

Text and Image Generation

- Text generation using attention and transformers
- Image generation

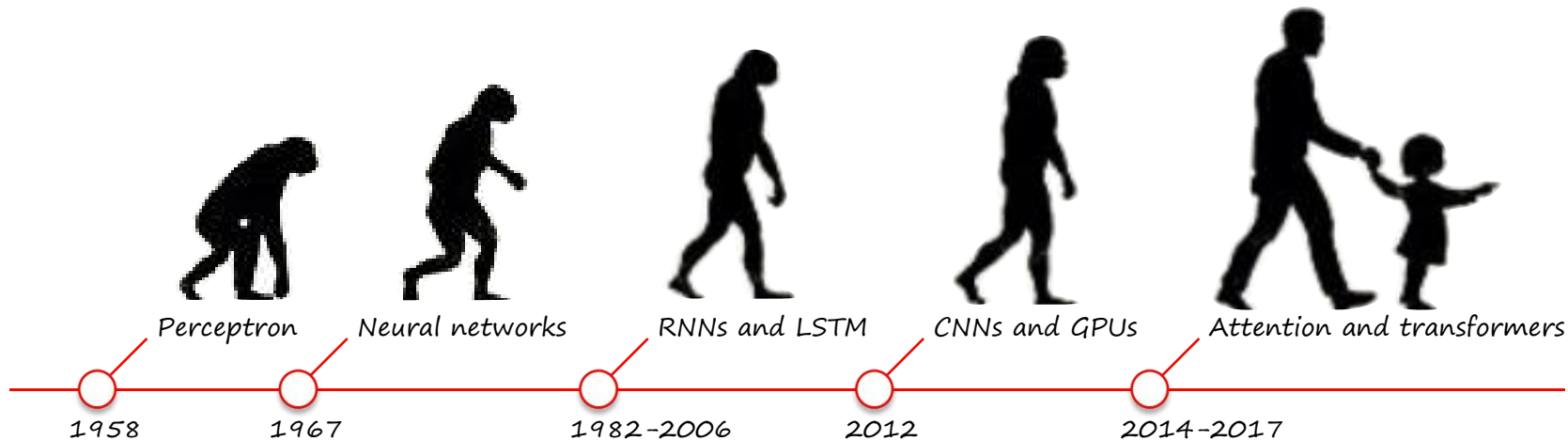


Text Generation using Attention and Transformers



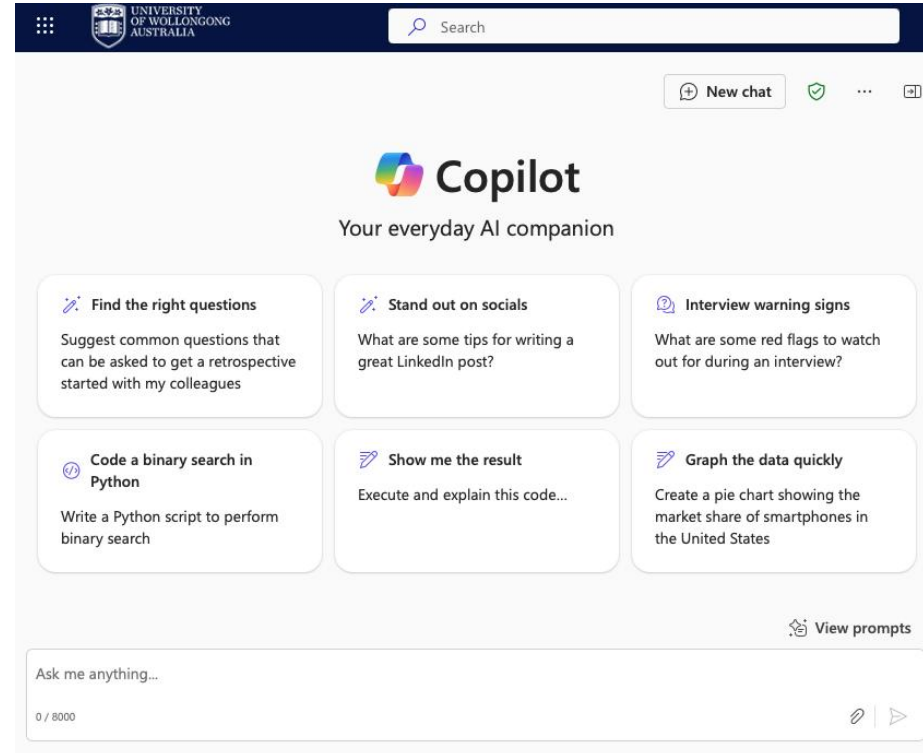
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Attention and transformers



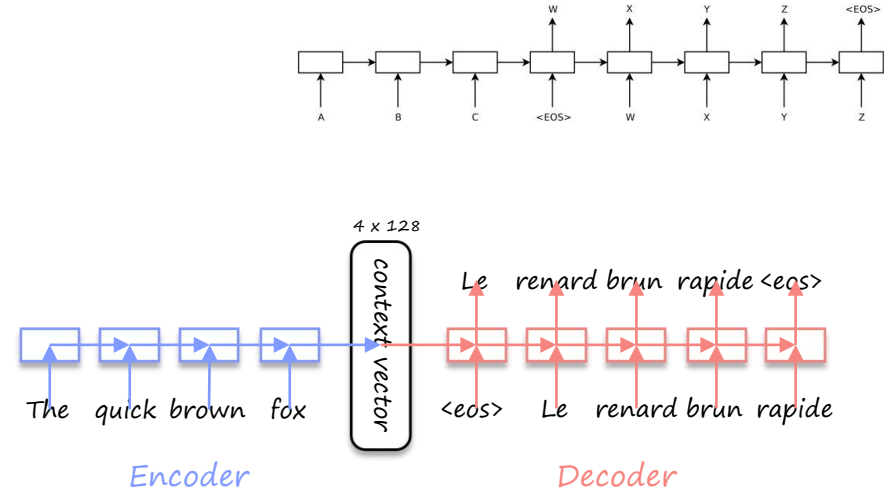
How we know Transformers

- GPT-4
 - ChatGPT
 - [Microsoft Copilot](#) (free for UOW)
 - [Duolingo Max](#)
- Gemini
 - Google search
 - Google Lens



Encoder-decoder architecture

- Sutskever et al. (2014)
- 'Sequence-to-sequence' learning for variable input-output lengths
- The Encoder maps the input to a limited dimensional space ('context vector'), e.g. 128 dimensions.
- The Decoder generates output (given the context).
- Uses LSTM layers per input/output word.

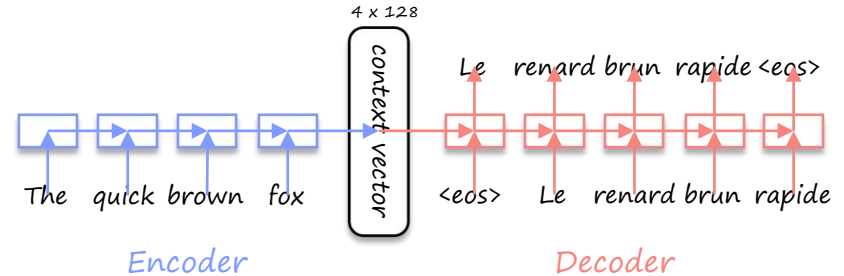


Limitation of autoencoders with only LSTMs

✗ Long-term dependency problem: long input sequences (e.g. whole paragraphs, pages) are challenging to understand.

Why?

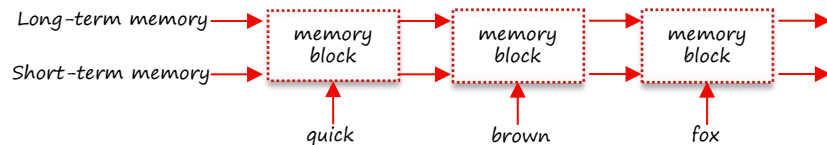
LSTMs take into account the whole input sequence, but not all input words are relevant to each output word.



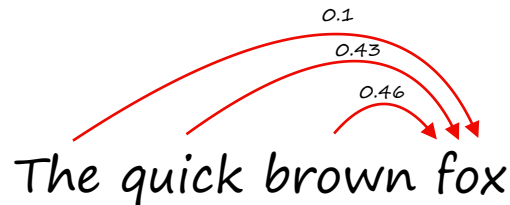
Attention

- Allows models to focus on specific parts of the input data.
- Enables the model to attend to different regions of the input sequence simultaneously.
- Improves performance by selectively weighing importance of certain tokens or features.

LSTM: All previous words matter



Attention: Some previous words matter

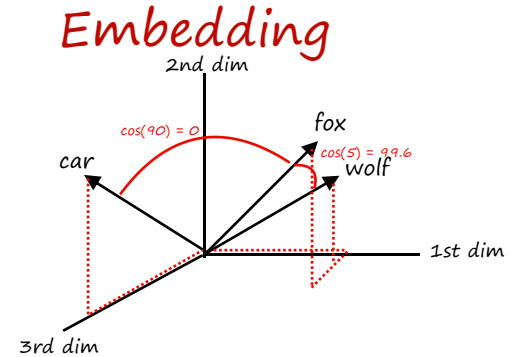


Word Embedding

- Represents the meaning of words based on a large number of texts.
- Words are represented as vectors in n-dimensional space
- (cosine) Angle or 'dot product' between vectors represents word similarity

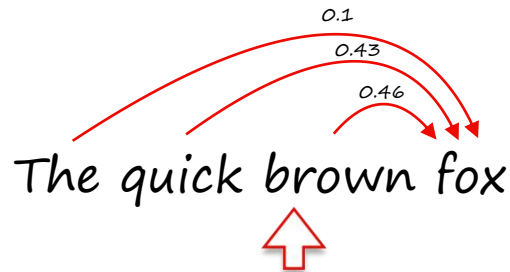
vector

$$\text{fox} = [\underset{1}{0.01} \quad \underset{2}{0.043} \quad \dots \quad \underset{n}{0.3}]$$



Self-attention for the encoder

- Bahdanau et al. (2014)
- Computes attention weights (i.e. context vector) for each token in the input sequence.
- Applies these weights to calculate a weighted sum of the input features for each token.
- Outputs a set of values, each representing the importance of a specific token in the input sequence to the target token.

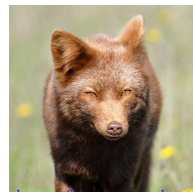


Attention

The quick brown fox

0	The	0.7	0.1
0	quick	0.3	0.43
0	brown	0	0.47
0	fox	0	0

Meaning is context-related!



[brown fox](#) by [matt knoth](#)

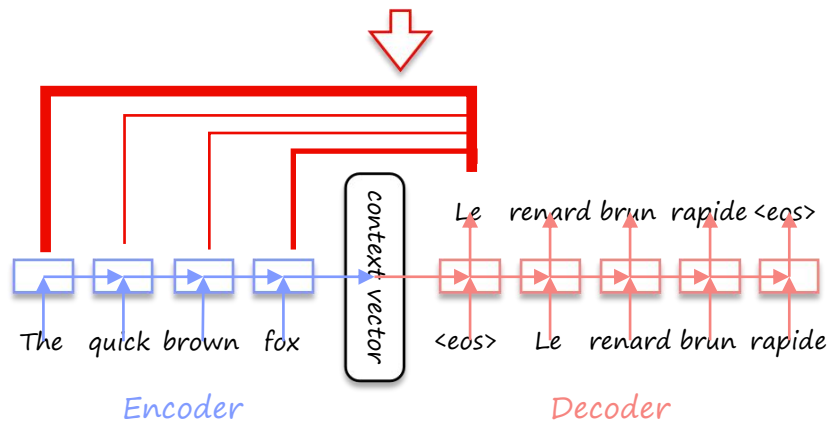
Self-attention for the decoder

- Bahdanau et al. (2014)
- Computes attention weights (i.e. context vector) for each token in the input sequence.
- Applies these weights to calculate a weighted sum of the input features for each token.
- Outputs a set of values, each representing the importance of a specific token in the input sequence to the target token in the output sequence.

Attention

Le renard brun rapide

0.9	0.1	0.05	0
0	0.9	0.05	0.8
0	0	0.85	0.1
0.1	0.9	0.05	0.1



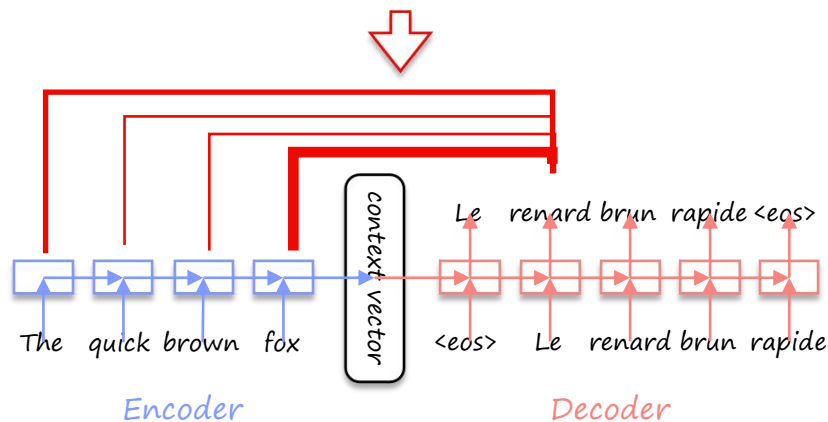
Self-attention for the decoder

- Bahdanau et al. (2014)
- Computes attention weights (i.e. context vector) for each token in the input sequence.
- Applies these weights to calculate a weighted sum of the input features for each token.
- Outputs a set of values, each representing the importance of a specific token in the input sequence to the target token in the output sequence.

Attention

Le renard brun rapide

0.9	0.1	0.05	0
0	0.9	0.05	0.8
0	0	0.85	0.1
0.1	0.9	0.05	0.1



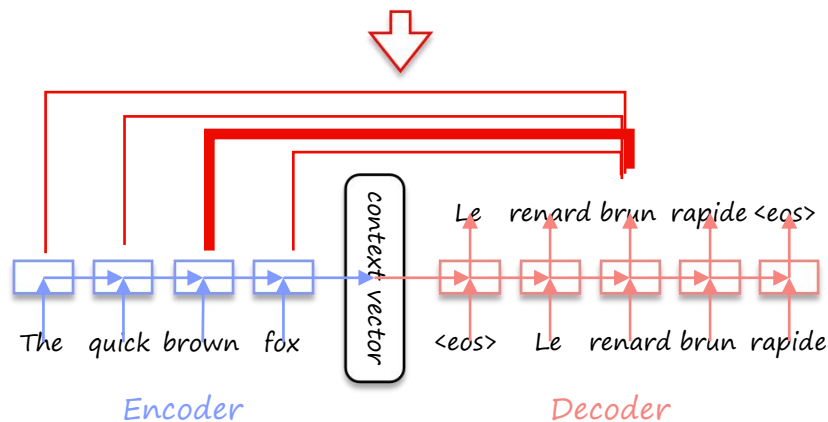
Self-attention for the decoder

- Bahdanau et al. (2014)
- Computes attention weights (i.e. context vector) for each token in the input sequence.
- Applies these weights to calculate a weighted sum of the input features for each token.
- Outputs a set of values, each representing the importance of a specific token in the input sequence to the target token in the output sequence.

Attention

Le renard brun rapide

0.9	0.1	0.05	0
0	0.9	0.05	0.8
0	0	0.85	0.1
0.1	0.9	0.05	0.1



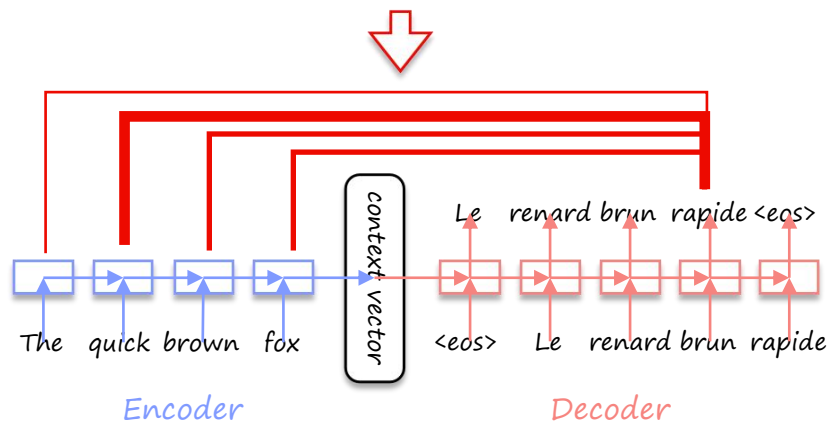
Self-attention for the decoder

- Bahdanau et al. (2014)
- Computes attention weights (i.e. context vector) for each token in the input sequence.
- Applies these weights to calculate a weighted sum of the input features for each token.
- Outputs a set of values, each representing the importance of a specific token in the input sequence to the target token in the output sequence.

Attention

Le renard brun rapide

0.9	0.1	0.05	0
0	0.9	0.05	0.8
0	0	0.85	0.1
0.1	0.9	0.05	0.1



Query, Key and Value

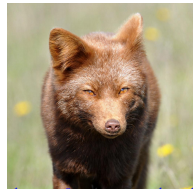
- Computes attention weights for each token in the input sequence.
- Applies these weights to calculate a weighted sum of the input features for each token.
- Outputs a set of values, each representing the importance of a specific token in the input sequence.

Attention

The quick brown fox

o	The	0.7	0.1
	quick	0.3	0.43
	brown	0	0.47
	fox	0	0

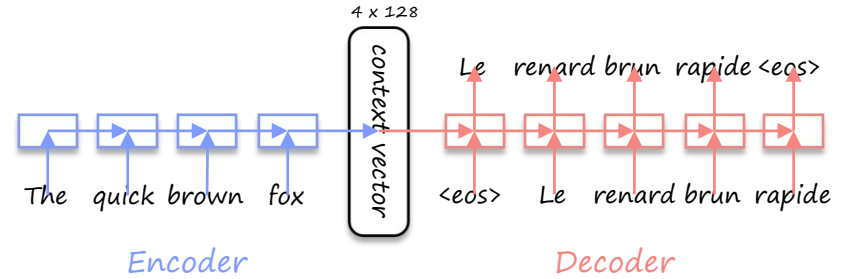
Meaning is context-related!



[brown fox](#) by [matt knott](#)

Limitation of single head attention

- ✗ Attention weights calculation time will increase dramatically with longer sequences.
- ✗ Limited capability to capture complex relationships between words.

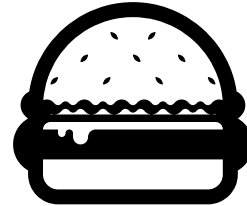


10-minute Break!

This will help you learn



10 minutes





What is the main limitation of LSTMs?

✗ *Long-term dependency problem: long input sequences (e.g. whole paragraphs, pages) are challenging to understand.*

Why?

LSTMs take into account the whole input sequence, but not all input words are relevant to each output word.

How does Attention solve the problems faced by LSTMs? (Select all that apply)

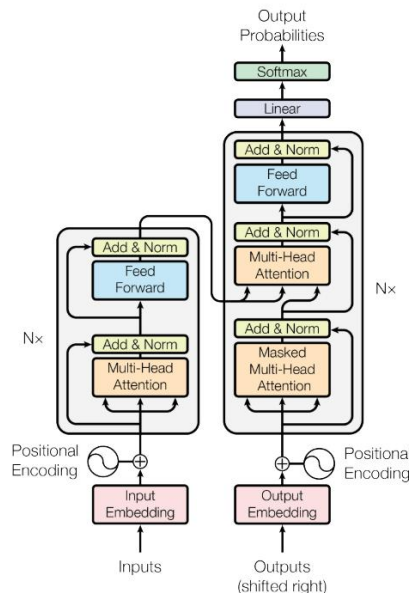
A. Alleviates the vanishing gradient problem by assigning weights to each part of the input.

B. Due to the weighting per input word, it can parallelise and enhance efficiency.

C. Allows models to focus on specific parts of the input data.

Transformer model

- 'Attention is all you need' (Vaswani et al., 2017)
- Uses self-attention instead of traditional recurrent or convolutional architectures.
- Encourages parallelisation by allowing multiple heads to attend simultaneously.



The quick brown fox

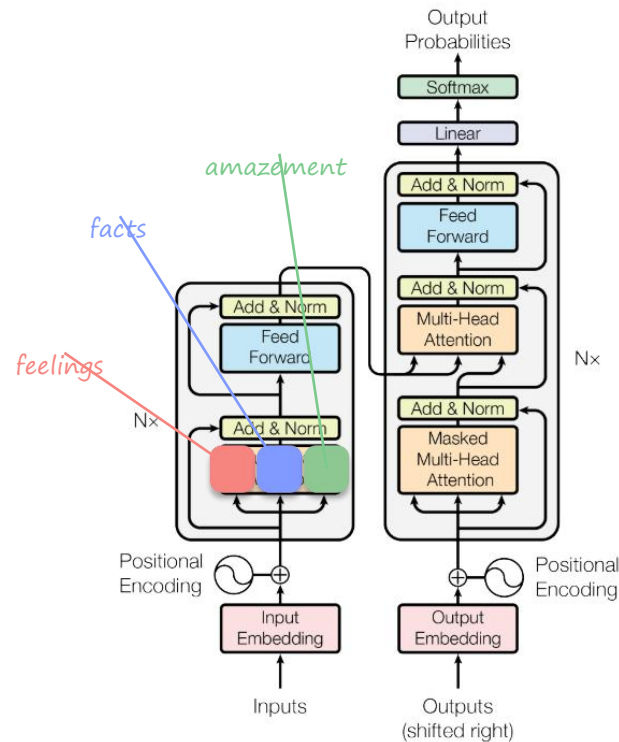
Le renard brun rapide

Multi-head attention

The following text has multiple types of relationships between words:

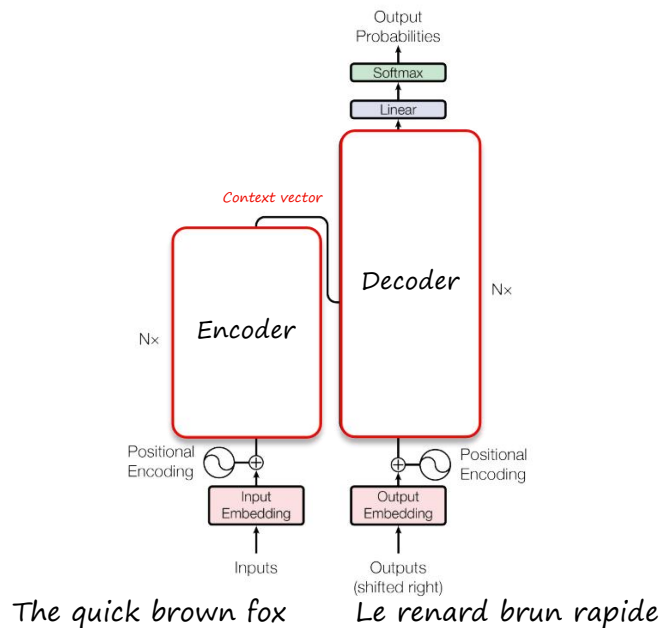
"As I stood at the edge of **Uluru** at dawn, *feeling the weight of my grandmother's whispered words* echoing in my mind - 'the land holds all your stories' - I couldn't help but think of the **500 million years** it took for the **sandstone monolith** to form, and the *sense of connection that washed over me as I gazed out at the rust-red hues* of the rocky terrain, feeling a *deep respect* for the traditional owners who had called this **sacred site** home for **thousands of generations**."

- Multi-head attention captures nuanced relationships



Transformer model

- 'Attention is all you need' (Vaswani et al., 2017)
- Uses self-attention instead of traditional recurrent or convolutional architectures.
- Encourages parallelisation by allowing multiple heads to attend simultaneously.
- Typically consists of an encoder and decoder, with a multi-layer architecture.



The Encoder maps the input to a limited dimensional space ('context vector').
The Decoder generates output (given the context).

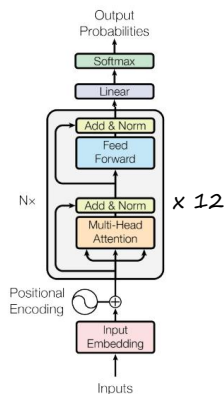
Advantages of Transformers

- ✓ Improved performance over traditional recurrent architectures.
- ✓ Ability to handle sequential data with variable-length inputs.
- ✓ No need for positional encoding or embedding.
- ✓ Can be parallelised easily using multiple GPUs (e.g. multiple attention heads).

Different types of transformers

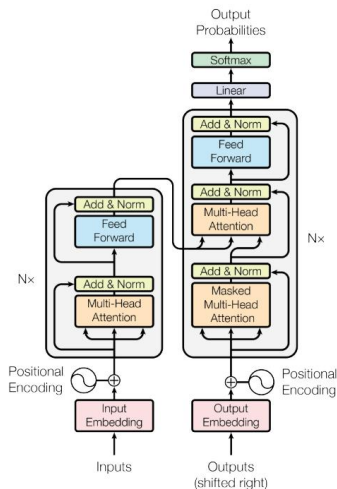
Encoder-only (e.g. BERT):

- Simpler architecture
- Only fixed input length
- Use: sentiment analysis.



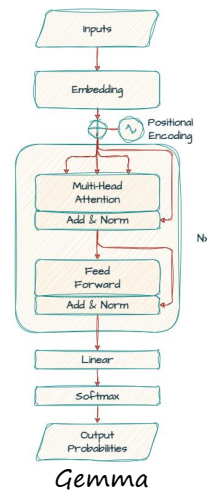
Encoder-decoder (e.g. T5, BART):

- Can handle longer sequences
- Can handle context
-



Decoder-only (e.g. GPT, Gemma):

- Simpler architecture
- Use: text generation, translation



Fantastic models and where to find them

- Hugging Face 🤖

<https://huggingface.co/>

- Gravity AI <https://www.gravity-ai.com/catalog>

- Google Model Garden

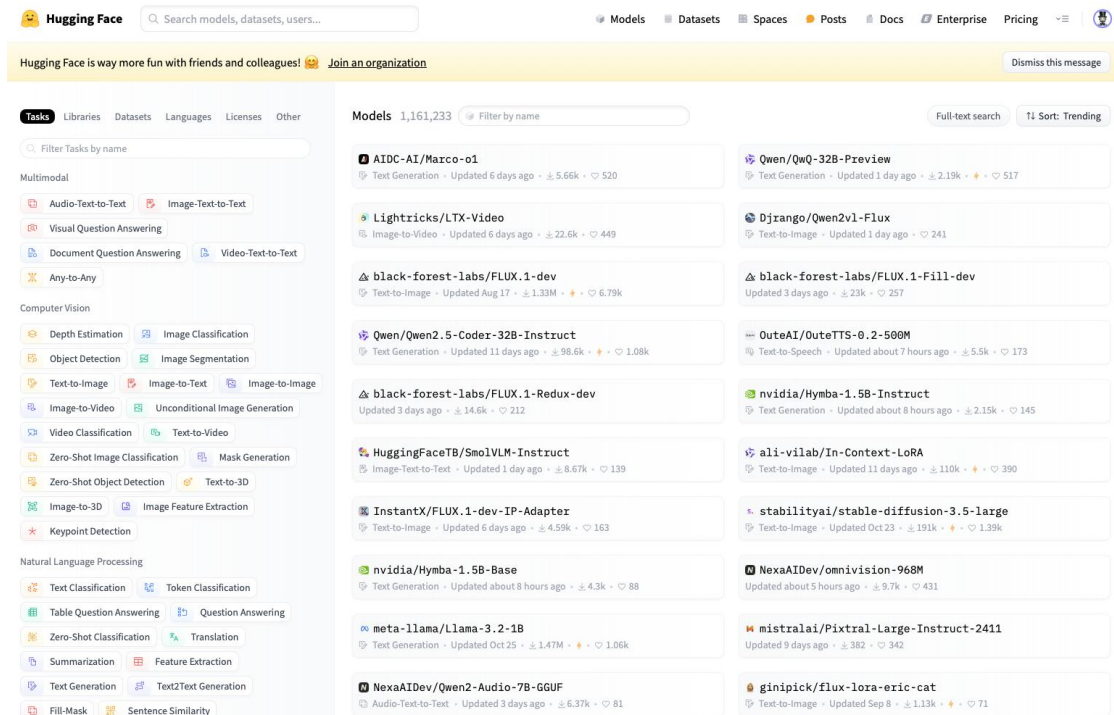
<https://cloud.google.com/model-garden?hl=en>

- Amazon Bedrock

<https://aws.amazon.com/bedrock/>

- Azure AI Model Catalog

<https://azure.microsoft.com/en-us/products/ai-model-catalog>



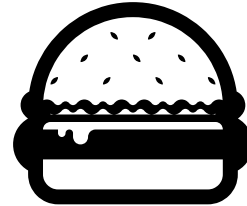
The screenshot shows the Hugging Face website interface. At the top, there's a search bar and navigation links for Models, Datasets, Spaces, Posts, Docs, Enterprise, and Pricing. Below the navigation bar, a yellow banner reads "Hugging Face is way more fun with friends and colleagues! 🤖 Join an organization". The main content area is divided into two columns. The left column, under the "Tasks" tab, lists various tasks like Multimodal, Computer Vision, and Natural Language Processing, each with sub-tasks. The right column, under the "Models" tab, displays a list of models with their names, descriptions, and statistics. The models listed include AIDC-AI/Marco-o1, Lightricks/LTX-Video, black-forest-labs/FLUX.1-dev, Qwen/Qwen2.5-Coder-32B-Instruct, Qwen/QwQ-32B-Preview, Djrange/Qwen2v1-Flux, black-forest-labs/FLUX.1-Fill-dev, OuteAI/OuteTTS-0.2-500M, nvidia/Hymba-1.5B-Instruct, ali-vilab/In-Context-LoRA, HuggingFaceTB/SmolVLM-Instruct, InstantX/FLUX.1-dev-IP-Adapter, nvidia/Hymba-1.5B-Base, meta-llama/Llama-3.2-1B, NexaAIdev/Qwen2-Audio-7B-GGUF, mistralai/Pixtral-Large-Instruct-2411, and ginipick/flux-lora-eric-cat.

7-minute Break!

This will help you learn



7 minutes



Match the following GenAI models and transformer types



BERT

T5

GPT

Decoder only

Encoder only

Encoder-Decoder

How does multi-head attention improve over single-head attention? (Select all that apply)



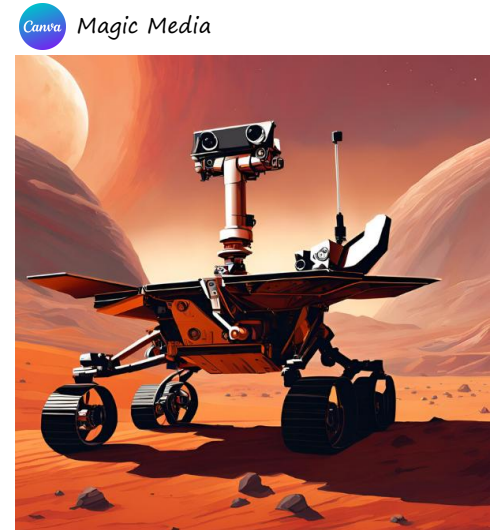
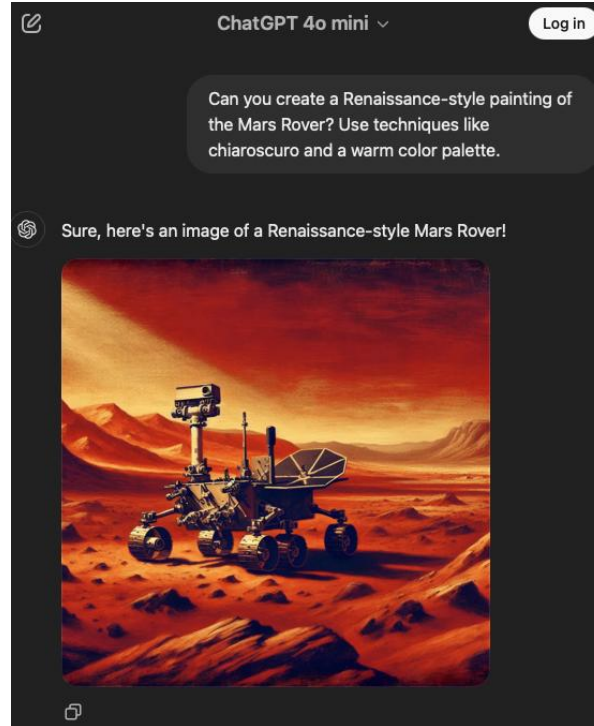
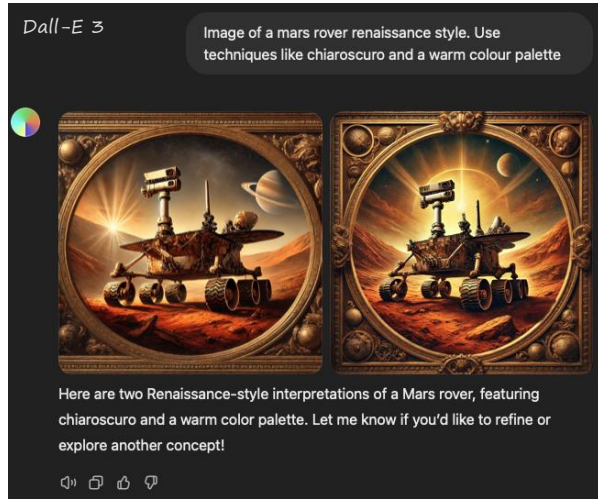
- A. Attention heads can run in parallel and enhance efficiency.
- B. They can capture complex relationships between words.
- ~~C. They can focus on different parts of a text.~~
- D. They take all previous words into consideration.

Image Generation part 1



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

How we know Image Generation



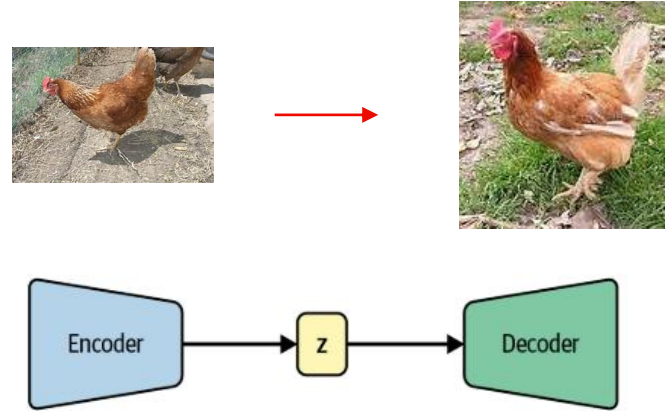
Variational Autoencoders (VAEs)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Variational Autoencoders (VAE)

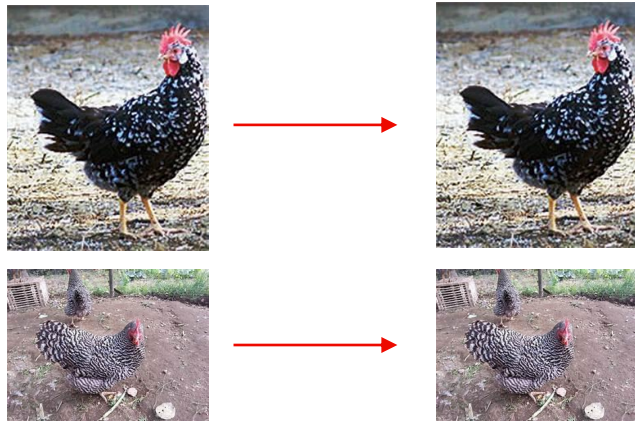
- By Kingma & Welling (2013, 2019)
- a neural network that is trained to perform the task of encoding and decoding an item, such that the output from this process is as close to the original item as possible (Foster, 2023).
- So the encoder models image characteristics ('embedding vector'), while the decoder generates images.



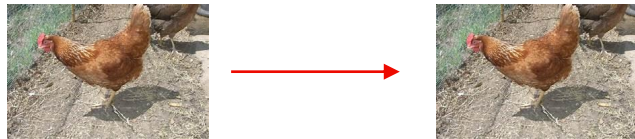
How autoencoders are trained

1. *Split data into train, test sets*

Training set



Test set



How autoencoders are trained

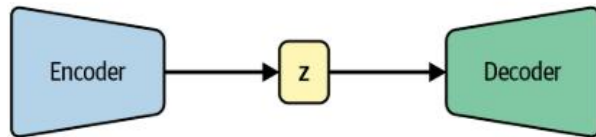
1. Split data into train, test sets
2. Build the encoder and decoder parts.

Table 3-1. Model summary of the encoder

Layer (type)	Output shape	Param #
InputLayer	(None, 32, 32, 1)	0
Conv2D	(None, 16, 16, 32)	320
Conv2D	(None, 8, 8, 64)	18,496
Conv2D	(None, 4, 4, 128)	73,856
Flatten	(None, 2048)	0
Dense	(None, 2)	4,098
Total params	96,770	
Trainable params	96,770	
Non-trainable params	0	

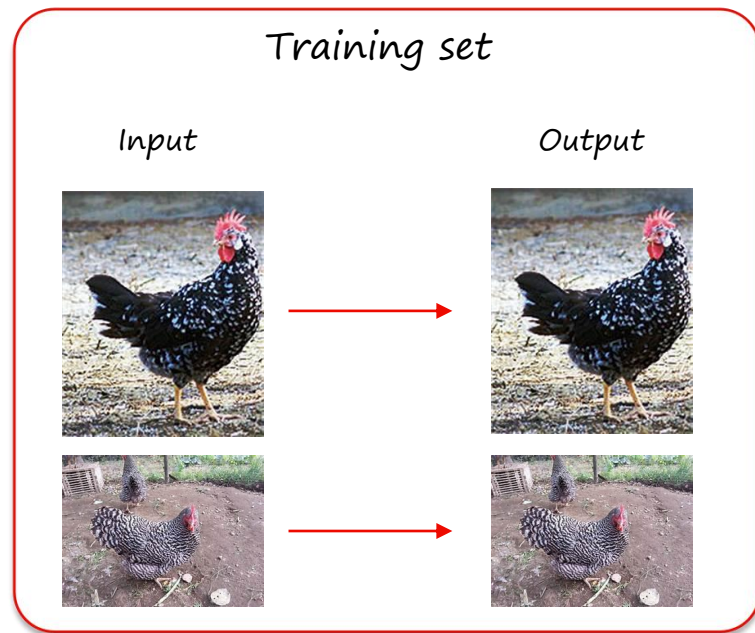
Table 3-2. Model summary of the decoder

Layer (type)	Output shape	Param #
InputLayer	(None, 2)	0
Dense	(None, 2048)	6,144
Reshape	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147,584
Conv2DTranspose	(None, 16, 16, 64)	73,792
Conv2DTranspose	(None, 32, 32, 32)	18,464
Conv2D	(None, 32, 32, 1)	289
Total params	246,273	
Trainable params	246,273	
Non-trainable params	0	



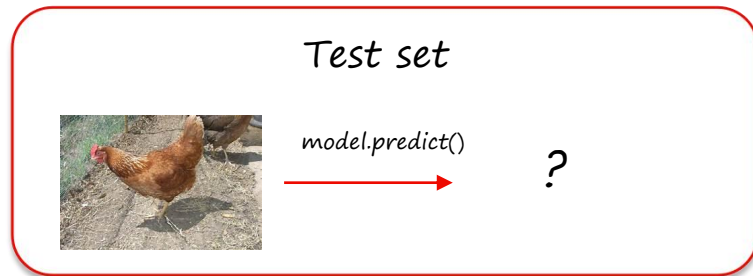
How autoencoders are trained

1. Split data into train, test sets
2. Build the encoder and decoder parts.
3. Train the auto encoder with the training set as input AND output.



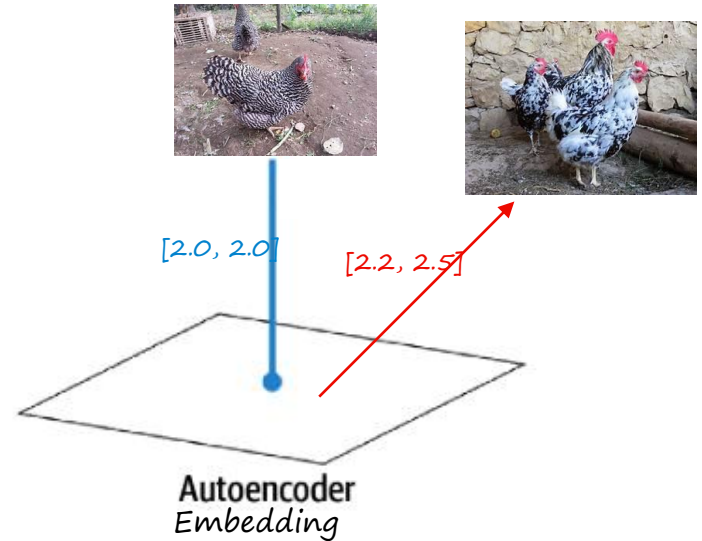
How autoencoders are trained

1. *Split data into train, test sets*
2. *Build the encoder and decoder parts.*
3. *Train the auto encoder with the training set as input AND output.*



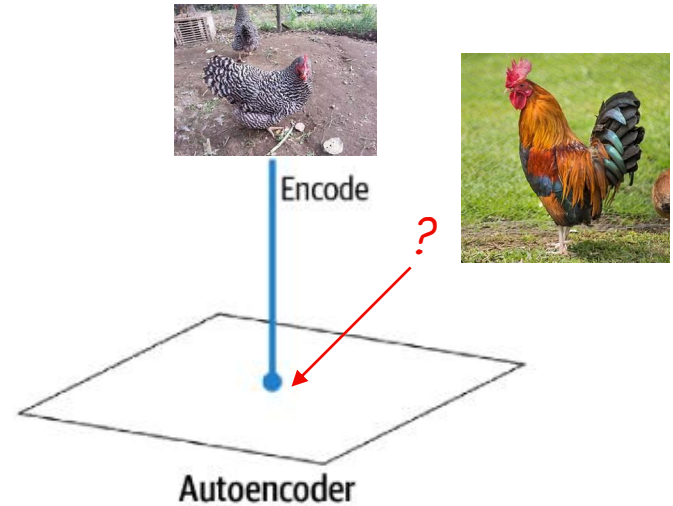
Then let the decoder generate

1. Define a vector, in this case $x = [2.2, 2.5]$ (two-dimensional 'embedding')
2. Let the decoder predict something:
`decoder.predict(x)`



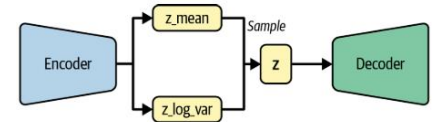
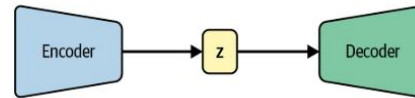
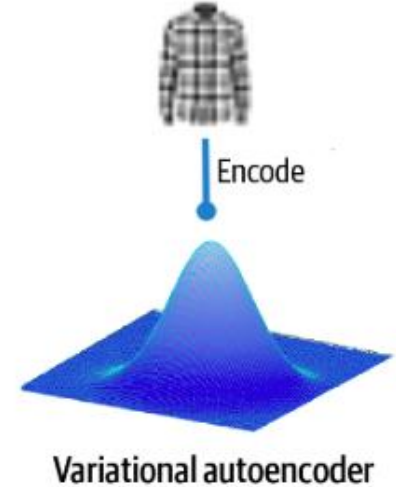
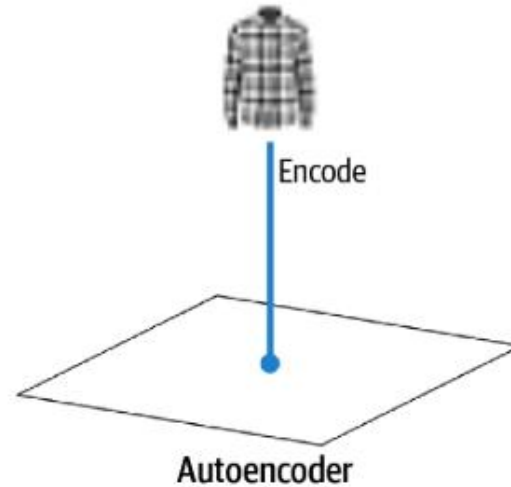
Limitation of autoencoders

✗ *Image encoding in a vector does not necessarily mean that the vector close to it results in a similar picture*



Variational Autoencoders

- Instead of modelling the learned image as a single spot, model it as a normal / Gaussian distribution.
- So instead of a single vector, we need a mean vector and variance vector.
- We then sample from the two vectors.

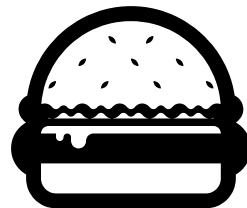


8-minute Break!

This will help you learn



8 minutes



What is the main goal of training a Variational Autoencoder (VAE)?



- A. To learn a deterministic mapping from input to output.
- B. To learn a probabilistic model that captures the distribution of input data.
- C. To perform image compression or feature extraction.
- D. To train a classifier for binary classification tasks.

Generative Adversarial Networks (GANs)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Generative Adversarial Networks (GANs)

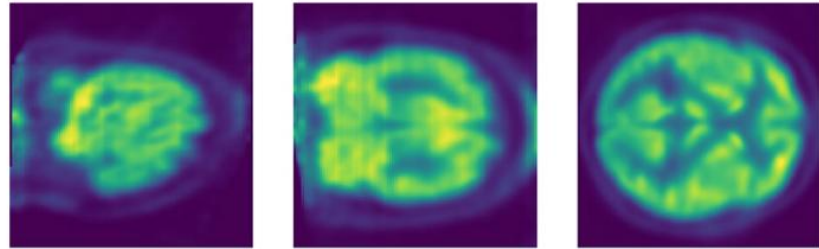
- Goodfellow et al., 2014
- GANs consist of two neural networks: a generator and a discriminator.
- The goal is to generate new data samples that are indistinguishable from real data samples.
- GANs can be used for unsupervised learning tasks, such as image generation, data augmentation, and anomaly detection.



(Karras et al., 2019)

Applications of GANs

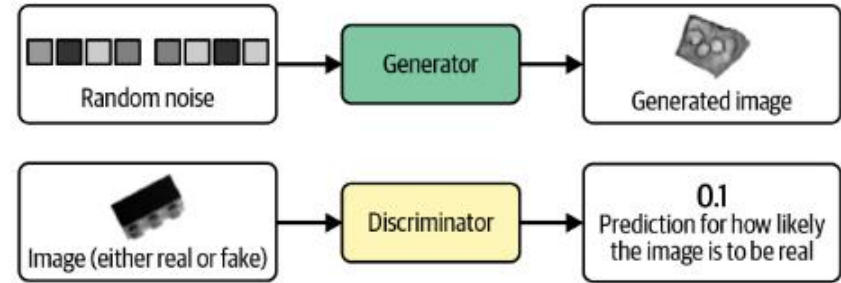
- Image generation (e.g., faces, objects)
- Data augmentation (e.g., for image classification tasks)
- Anomaly detection (e.g., identifying outliers in data)
- Style transfer and colorisation
- Text-to-image synthesis



(Islam & Zhang, 2020)

GAN architecture

- *Generator learns to produce realistic data samples by minimising the loss function*
- *Discriminator learns to correctly classify real and fake samples by maximising the loss function*



(Foster, 2003, p.97)

Training a GAN

- *GAN training involves alternating between generator and discriminator updates.*
- *The Generator G tries to maximise $D(G(z))$ to 1 (i.e. 'real'), where z is a latent vector.*
- *The Discriminator D tries to minimise $D(G(z))$ to 0 (i.e. 'fake').*

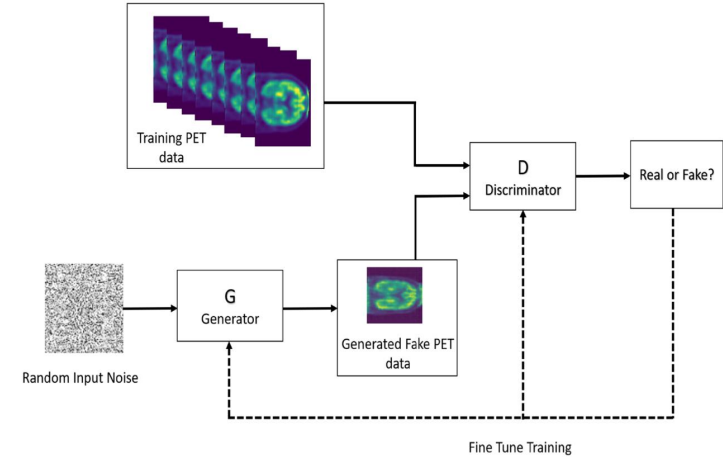


Fig. 2 Proposed synthetic brain PET image generator

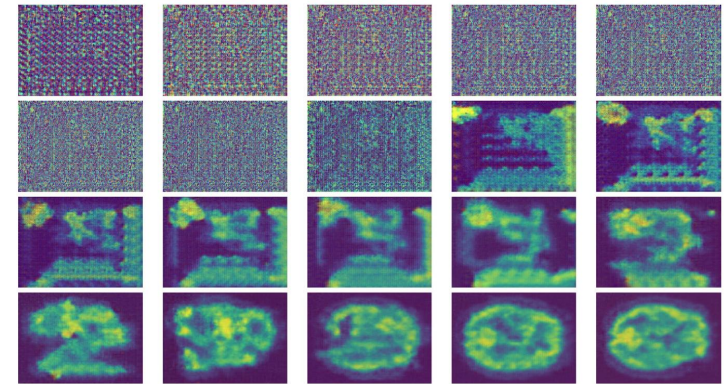
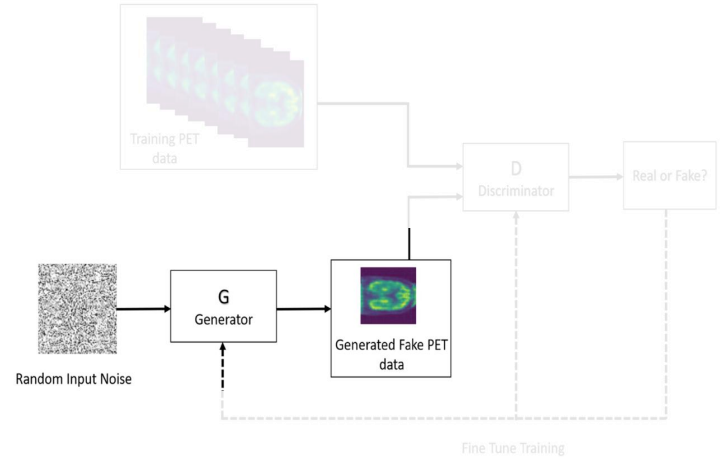


Fig. 4 Visualization of the generator output in the training process

How a GAN generates images

- *Get the Generator's architecture and weights.*
- *Choose a random noise vector as input.*
- *Generate the image.*



Limitations of GANs

- ✗ *Mode collapse: the generator produces limited variations of the same output.*
- ✗ *Vanishing gradients: the generator's weights become too small during training, causing the generator to lose its ability to learn*
- ✗ *Unstable training: the discriminator becomes too powerful and causes the generator to converge to a poor solution*

Wrap-up



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Summary

- Attention and Transformer architectures are pivotal to Generative AI.
- Different types of transformers:
 - Encoder-only: sentiment analysis
 - Encoder-Decoder: translation
 - Decoder-only: generation
- Two architectures for image generation:
 - Variational Autoencoders (VAEs) that encode images to low-dimensional space and sample images from a Gaussian distribution.
 - Generative Adversarial Networks (GANs) that use a generator to create fake images and discriminator to detect fake images.

For next lecture

- *Read Poldrack, Lu, & Begus, 2023 about AI-assisted coding*

Take Home Message...

*Consider the message that you want to create when
designing your interfaces.*



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Things to think about

What is the purpose of prototyping?

*What is the difference between Lo-fi
and Hi-fi designs?*

*What is a storyboard? When can
these be used?*

What should you show your client?

References and Resources

- Bahdanau, D., Cho, K., & Bengio, Y. (2014, September 1). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR2015. <https://doi.org/https://doi.org/10.48550/arXiv.1706.03762>
- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative Adversarial Networks. 1–9. <https://doi.org/10.1001/jamainternmed.2016.8245>
- Islam, J., & Zhang, Y. (2020). GAN-based synthetic brain PET image generation. Brain Informatics, 7(1). <https://doi.org/10.1186/s40708-020-00104-2>
- Karras, T., Laine, S., & Aila, T. (2019). A Style-Based Generator Architecture for Generative Adversarial Networks. <http://arxiv.org/abs/1812.04948>
- Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. CoRR, abs/1312.6114.
- Kingma, D. P., & Welling, M. (2019). An Introduction to Variational Autoencoders. Foundations and Trends in Machine Learning, xx(xx), 1–18. <https://doi.org/10.1561/XXXXXXXXXX>
- Poldrack, R. A., Lu, T., & Beguš, G. (2023). AI-assisted coding: Experiments with GPT-4. <http://arxiv.org/abs/2304.13187>
- Sutskever, I., Vinyals, O., & Le, Q. v. (2014). Sequence to Sequence Learning with Neural Networks. Proceedings of the 27th International Conference on Neural Information Processing Systems – Volume 2 (NIPS'14), 3104–3112. <http://arxiv.org/abs/1409.3215>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017, June 12). Attention Is All You Need. 31st Conference on Neural Information Processing Systems (NIPS 2017). <http://arxiv.org/abs/1706.03762>

Questions



UNIVERSITY
OF WOLLONGONG
AUSTRALIA